APEXFORMZ V2

Product Requirements Document

Modular human motion intelligence and sport expert engines

A shared motion foundation for independently evolving sport and movement analysis, real-time coaching, and athlete progression.

Foundation release focus

Migrate the existing Taekwondo pipeline into isolated engines, preserve responsive feedback, and establish trustworthy scoring, reference data, and versioned results.

Document control	Value
Product	ApexFormz / SmartCoach
Version and date	2.0 | September 2026
Document status	Master Product and Engineering PRD
Review audience	Product, engineering, data, quality, and coaching
Product and technical owners	To be assigned
Release date and approvers	To be assigned

Decision required

Confirm foundation scope, assign ownership, and resolve the acceptance and contract decisions listed in Section 20. Proposed additions are identified throughout the document.

Contents

01 Product overview 3
02 Scope and priorities 4
03 Users and experience 5
04 Existing platform and migration baseline 6
05 Target architecture 7
06 Perception and human motion 8
07 Sport and movement engines 9
08 Phases and reference retrieval 10
09 Reference data governance 11
10 Shared analysis and scoring 12
11 Coaching and confidence 13
12 Training flow and failure handling 14
13 Real time performance 15
14 APIs and integration 16
15 Persistence and athlete intelligence 17
16 Session state and operations 18
17 Migration and rollout 19
18 Validation and success metrics 20
19 Release acceptance 21
20 Risks and open decisions 22
Appendix A Motion contracts 23
Appendix B Session and result contracts 24
Appendix C Package and code organization 25
Appendix D Illustrative policies and glossary 26

Reading guide

Sections 01–05 establish the product and architecture. Sections 06–16 define requirements and operations. Sections 17–20 cover migration, validation, acceptance, and open decisions. Appendices retain engineering contracts and illustrative configurations.

01 Product overview

ApexFormz V2 establishes a modular sports coaching platform built on a shared Human Motion Engine and independently managed Sport and Movement Expert Engines. The foundation release migrates the existing Taekwondo analysis into this structure while preserving real-time feedback, the application stack, authentication, and training engagement features.

The architecture separates body measurement from sport interpretation. A joint configuration may look similar in a badminton smash, a tennis serve, or a cricket throw, yet require different phase definitions, reference examples, scoring weights, and coaching. V2 must prevent data or rules from one sport from influencing another sport's analysis.

Product objectives

- Establish a sport-neutral motion representation with consistent geometry, timing, confidence, and optional depth data.
- Restrict analysis to the selected sport, movement, and phase, with independent reference datasets and scoring policies.
- Reuse proven analysis capabilities through shared services instead of duplicating motion mathematics in each sport package.
- Preserve responsive live feedback while keeping expensive post-repetition models outside the live frame path.
- Make a new sport primarily a package, configuration, reference-data, and validation effort.
- Store structured repetition results that can support athlete trends and personalized coaching over time.

Outcomes and evidence

Outcome	Evidence required
Domain isolation	Automated tests record zero access to reference repositories belonging to unselected sports.
Extensibility	A second sport uses the engine interface without sport-specific changes to the Human Motion Engine.
Responsive coaching	AI latency meets the stated P50 and P95 targets; live feedback is measured end to end.
Reproducible analysis	Results retain movement and reference versions; richer engine and policy provenance is progressively added.
Trustworthy feedback	Low-confidence analysis abstains or reduces specificity; unavailable AQA never produces a fabricated score.

Intended readers and use

Product, engineering, data, and coaching reviewers use this PRD to align release scope, implement interfaces, prepare datasets, and evaluate acceptance evidence. Taekwondo is the reference implementation. Badminton is the recommended candidate for the second sport; that selection remains open.

Requirement language. "Must" and "shall" identify requirements within the stated delivery phase. "Should" identifies a recommendation. Items marked "Proposed" expand product or operational detail and require review before becoming commitments. Numerical examples illustrate contracts or policies and are not validated thresholds. P0–P3 describe priority, not delivery dates.

02 Scope and priorities

The foundation release focuses on architectural isolation and reliable migration. Structured rep data prepares for personalization, while trend-based coaching and a second native sport remain subsequent work.

Priority	Capability	Scope
P0	Shared motion core and contracts	Normalize motion, preserve z, centralize confidence and filtering, define engine interfaces and registries.
P0	Reliability	Handle unavailable AQA, session lifecycle, version diagnostics, and bounded frame processing.
P1	Taekwondo migration	Package current movements, phase definitions, references, models, and movement scoring.
P1	Dataset architecture and validation	Introduce manifests, immutable versions, phase labels, expert metadata, golden datasets, and coach review.
P2	Athlete intelligence	Add recurring-issue tracking, movement profiles, consistency, and progression metrics.
P2	Second sport	Prove extension through the plugin interface; badminton is a candidate.
P3	Recognition and equipment	Add confidence-gated sport or movement classification and optional object perception.

Initial V2 boundaries

Initial input sources are webcam and uploaded video. The browser MediaPipe implementation remains the default perception path. Users explicitly select sport and movement. Existing static and dynamic analysis must continue to operate during migration.

The initial migration does not require replacing MediaPipe, Next.js, Node/Express, or FastAPI; rewriting gamification; training one universal sports model; recognizing every sport or movement automatically; supporting multi-person tracking or native mobile apps; automating coach certification; or tracking all balls, shuttles, rackets, and bats in real time.

Future extension points

Additional capture sources may include mobile, external, and multiple cameras and prerecorded coach footage. Advanced reference indexes may replace brute-force search behind a stable interface. A learned Human Motion Encoder may feed separate sport heads and, where useful, movement heads. This shares motion representation while preserving sport-specific decision boundaries.

Future object perception may expose a TrackedSportObject containing type, position, optional velocity, and confidence. Sport packages may declare optional or required equipment observations. These interfaces are extension points, not initial equipment-tracking commitments.

03 Users and experience

The primary experience is an athlete selecting a movement, setting up the camera, performing repetitions, and receiving a small number of useful corrections. Coaches and movement specialists define valid references and review the quality of analysis. Engineers maintain shared capabilities and add isolated sport packages.

Proposed user stories and acceptance detail

ID	User need	Observable acceptance
UX01	As an athlete, I want to train the movement I selected.	Sport and movement remain visible; session setup validates the selection; feedback identifies the same sport and movement.
UX02	As an athlete, I want clear camera setup guidance.	Poor tracking produces actionable setup guidance and suppresses precise biomechanical corrections until evidence improves.
UX03	As an athlete, I want to understand a completed rep.	A rep summary exposes overall score, available components, confidence, and the main detected issues. Unavailable components are identified.
UX04	As a coach, I want trustworthy reference material.	Dataset records identify movement, version, camera context, annotations, and review status. Published versions are immutable.
UX05	As an engineer, I want independent sport packages.	Disabling a sport leaves other sports operational; isolation tests and declared feature requirements pass.

Live coaching interface

Preserve the webcam panel, body skeleton, live score, issues, phase, and rep count. Add a sport indicator, movement indicator, and engine state: Ready, Tracking, Movement detected, or Analyzing. Confidence warnings should tell the athlete what to change, such as moving farther from the camera to bring the body into view.

The live display should present one primary correction and no more than one secondary correction. A detailed component breakdown belongs in the rep or session summary. Feedback must stabilize across frames rather than flash after one noisy observation.

Proposed uploaded video journey

The athlete selects sport, movement, dominant side, and camera view, then chooses a video. The interface validates the supported input, shows processing progress, and provides repetition summaries and time-linked issues when analysis completes. Cancellation, unsupported media, and processing failure receive explicit states. File formats, duration and size limits, storage location, and processing-time objectives remain decisions to resolve before delivery.

Video analysis should preserve media timestamps and the same domain and confidence boundaries as live analysis. Define its sampling policy separately; live frame-dropping behavior must not silently determine offline processing.

04 Existing platform and migration baseline

ApexFormz already combines browser pose extraction with real-time orchestration and an AI analysis service. V2 reorganizes these capabilities around explicit ownership boundaries. The baseline described here is the current product architecture to preserve during migration.

Component	Existing responsibilities	V2 treatment
Frontend	Next.js interface, webcam capture, MediaPipe Pose Landmarker, authenticated Socket.IO, skeleton and feedback display	Retain the stack and interaction model; extend session context and result presentation.
Backend	Node/Express authentication, sessions, static or dynamic routing, frame relay, persistence, XP, streaks, leaderboards, and Circles	Retain orchestration and compatibility; preserve newest-frame backpressure and keep-alive connections.
AI service	FastAPI, static pose analysis, dynamic movement analysis, reference matching, scoring, and post-rep inference	Extract shared motion services and route through isolated sport and movement engines.
Shared data	TypeScript types and PostgreSQL persistence	Standardize frame contracts and add structured repetition and version records.

Capabilities to reuse

Retain and reorganize BaseAnalyzer, AnalyzerRegistry, DynamicAnalyzer, PhaseDetector, PyomecaStreamAdapter, the One Euro KeypointSmoother, FrameMatcher, KineticChainValidator, TrajectoryScorer, MovementRegistry, ST-GCN Action Quality Assessment, expert-frame data, and movement blueprints.

DynamicAnalyzer currently coordinates readiness, normalization, filtering, phase detection, scoring, trajectory recording, fluidity, confidence, and overall scoring. It may remain as a temporary adapter while the generic operations move into the core and movement-specific orchestration moves into MovementEngine.

Compatibility constraints

- Keep `/analyze` and `/analyze/dynamic` operational during migration.
- Retain `session:start`, `frame:submit`, `feedback:result`, and `session:stop` events.
- Preserve existing authentication, gamification, social training, and static analysis behavior.
- Keep the `frameInFlight` and latest `pendingFrame` strategy; do not accumulate an unbounded queue.
- Preserve current behavior during core extraction and avoid major scoring changes while introducing engine interfaces.

The application services remain `apps/frontend`, `apps/backend`, and `apps/ai-service`. The refactor primarily changes how analysis code, state, configuration, and datasets are organized within those boundaries.

05 Target architecture

The Human Motion Engine owns sport-independent measurements. Sport and Movement Expert Engines interpret those measurements within an explicit context. Shared analysis services execute reusable algorithms with configuration supplied by the selected movement.

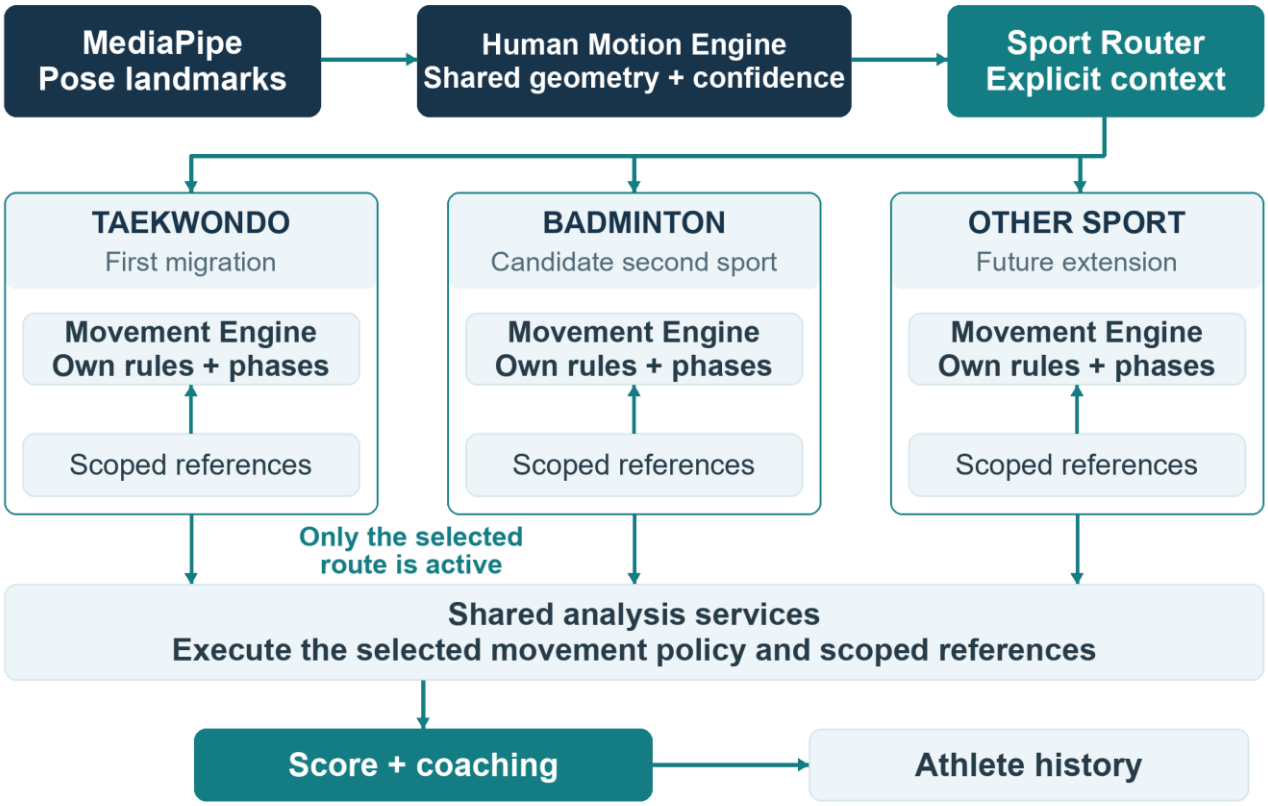


Figure 1 Shared motion processing with isolated sport and movement interpretation

Architectural requirements

ARC01. The core shall expose normalized motion and confidence without sport-specific terminology, reference selection, scoring rules, or coaching conclusions.

ARC02. Only the selected sport and movement may supply references, phase definitions, weights, models, and rules to a session. Shared analysis services must preserve that context.

ARC03. Each sport shall support independent registration, versioning, testing, and disabling. Adding a sport should not change core logic unless a genuinely universal feature is needed.

For example, the core may measure hip and shoulder angular velocity. A badminton movement engine may identify an ordering issue. The Coaching Engine converts that issue into a correction, and the Athlete Intelligence layer later evaluates whether it recurs over multiple repetitions.

06 Perception and human motion

MOT01. The perception layer shall produce timestamps, frame identity, named landmarks, detection confidence, and source width, height, and estimated frame rate. Landmark records contain x, y, visibility, and z whenever the detector provides depth.

MOT02. The Human Motion Engine shall standardize 2D and 3D representation and compute available sport-neutral measurements, with appropriate confidence. An absent depth value must not be silently treated as measured depth.

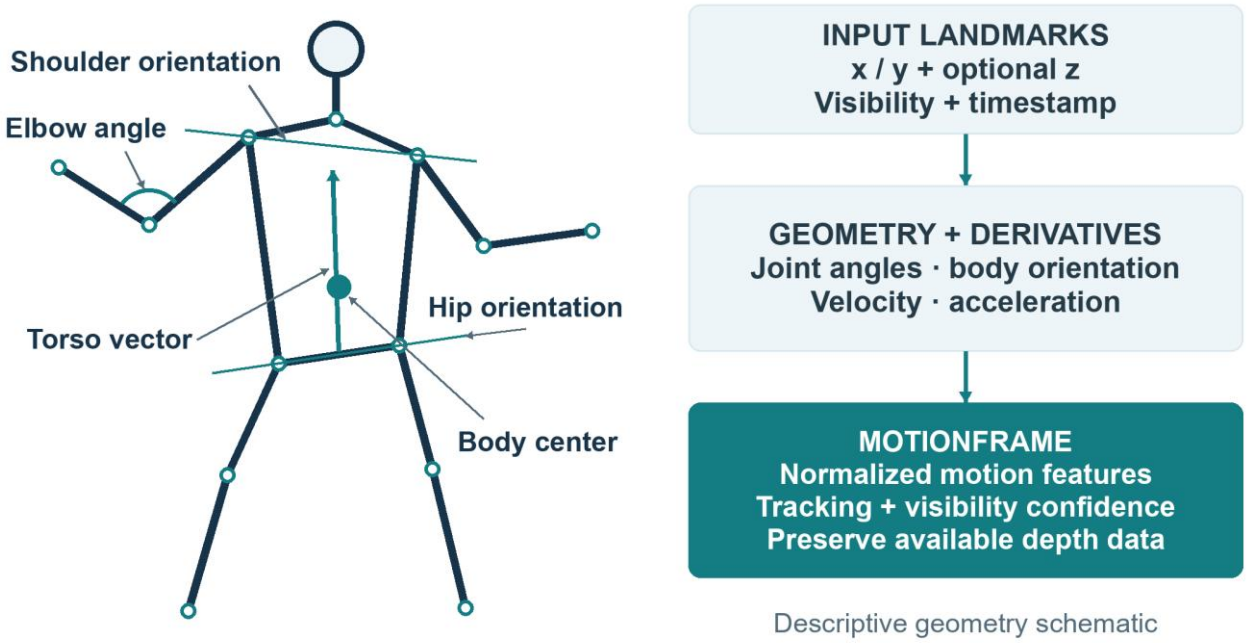


Figure 2 Schematic geometry and motion features derived from pose observations

Feature family	Required capability where technically supported
Geometry	Joint positions; body center; shoulder and hip widths; limb lengths; torso vector; elbow, shoulder, hip, knee, and ankle angles.
Motion	Position and angular velocity; acceleration; angular acceleration; jerk; torso inclination and shoulder or hip rotation.
Temporal and orientation	Activity, history, onset, termination, approximate rep boundaries, body-facing direction, stance direction, and left or right orientation.

MOT03. Centralize One Euro filtering and the optional Pyomeca/Butterworth pipeline, derivatives, and interpolation. A sport-specific smoothing exception requires a documented reason.

MOT04. A Feature Registry shall allow engines to request the features they use. MotionFrame is the common input to sport engines. Coordinate units, feature identifiers, time conventions, and per-feature confidence representation must be resolved in the contract review.

07 Sport and movement engines

ENG01. The Sport Router shall resolve the user-selected sport through SportEngineRegistry. Only that engine participates in scoring. Explicit movement selection is the initial default within the selected sport.

Ownership boundaries

Layer	Owns	Excludes
Sport Engine	Terminology, movement inventory, sport rules, reference datasets, allowed phase models, classification extension, score policies, sport coaching, models, and equipment requirements	Generic landmark extraction, angle mathematics, signal filtering, authentication, and persistence
Movement Engine	Movement phase model, required features, reference constraints, feature weights, kinetic chain, scoring policy, rep analysis, and coaching rules	Reference or rule access outside its sport and movement
Shared services	Reusable phase, matching, trajectory, kinetic-chain, fluidity, and AQA implementations	Global assumptions that impose one sport's interpretation on another

Engine interfaces

The SportEngine interface provides `supported_movements()`, `create_session(config)`, `classify_movement(motion_window)`, `get_movement_engine(movement_name)`, and `required_features()`. The classification method is an extension point; manual-only engines need a documented unsupported result until automatic recognition is introduced.

MovementEngine provides `movement_id`, `required_features()`, `process_frame(motion_frame)`, `detect_phase(motion_frame)`, `score_frame(motion_frame)`, `complete_rep()`, and `generate_analysis()`.

ENG02. Movement engines shall be able to use shared analysis modules without assuming a universal scoring formula, phase sequence, joint-weight table, or model.

Context and future recognition

ENG03. Store dominant side as left, right, or unknown. Mirroring must be movement-aware. A left-handed execution is not automatically interchangeable with every right-handed reference condition.

ENG04. Sessions should identify camera view. The proposed directional vocabulary is front, front-left, left, rear-left, rear, rear-right, right, front-right, or unknown. Legacy "side" labels need an explicit mapping.

Future automatic sport or movement recognition must return confidence and support an uncertain state. It must not silently route ambiguous input. Recognition thresholds are not specified by this PRD and require calibration.

08 Phases and reference retrieval

REF01. Reuse the finite-state-machine PhaseDetector as a configurable framework. Each movement owns its phase definition and transition logic; the shared framework must not impose one universal phase model.

Example movement	Illustrative phase sequence
Badminton smash	Ready → Preparation → Loading → Acceleration → Contact → Follow-through → Recovery
Taekwondo kick	Stance → Chamber → Extension → Rechamber → Return

REF02. Reference retrieval shall constrain sport, movement, and current phase before nearest-neighbour comparison. Compatible camera view, dominant side, and athlete-profile range may further constrain retrieval.

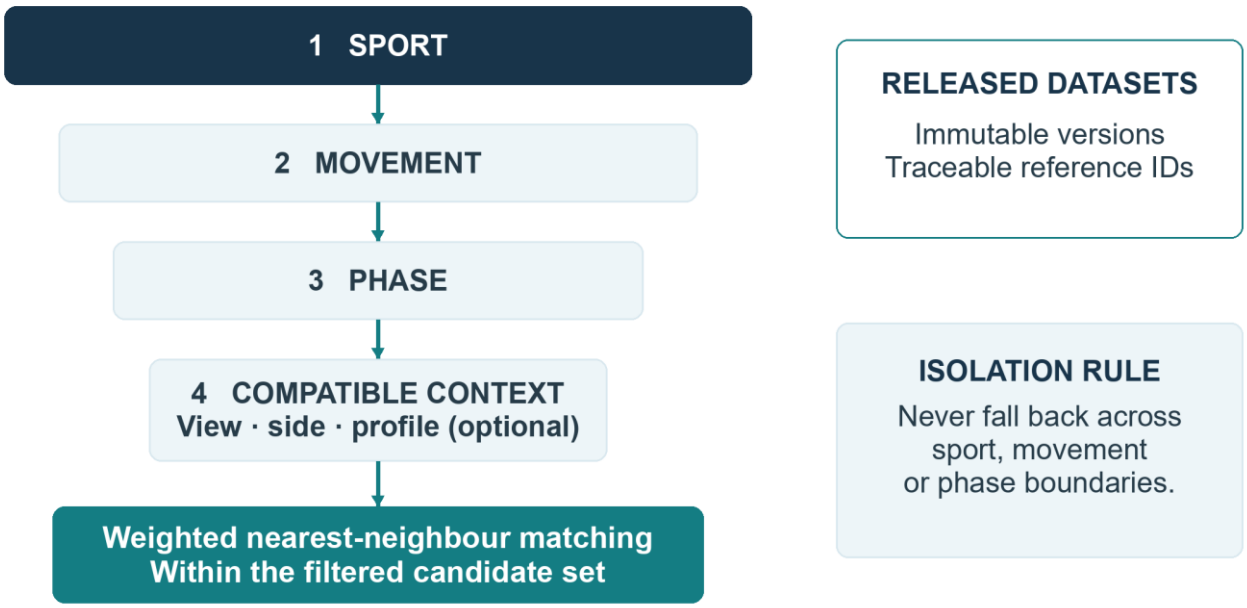


Figure 3 Reference selection narrows the comparison domain before similarity search

Retrieval behavior

An athlete in the acceleration phase of a badminton smash must be compared with acceleration references for that movement. Search must not widen into other sports, movements, or phases to obtain a convenient match.

Proposed clarification. If the eligible reference set is empty or unusable, return an explicit unavailable comparison state. Preserve measured motion and tracking guidance, and withhold reference-dependent scores until valid evidence exists.

REF03. Begin with a BruteForceIndex behind a stable search interface. KDTree, HNSW, or FAISS may replace it as scale requires. Movement engines should not depend on the indexing implementation. Measure reference-search latency before selecting an optimization.

09 Reference data governance

DAT01. Every movement dataset shall have a manifest and an immutable release version. Sessions must record the dataset version used to produce their scores. New data or labels produce a new version rather than changing a released dataset in place.

Organization and metadata

Organize reference data by sport and movement, with compatible camera views and phase indexes. A movement dataset may include `manifest.json`, view-specific recordings or frames, `phase_index.json`, `trajectories.json`, and `phase_centroids.json`.

Metadata group	Required or planned content
Dataset identity	Sport, movement, version, camera views, dominant sides, and phase vocabulary
Dataset inventory	Athlete and recording counts; sample and recording identifiers
Expert quality goals	Anonymized athlete ID, expertise classification, recording camera, movement type, dominant side, and recording quality
Review and labels	Valid or invalid sample label, phase annotations, coach approval, and reference confidence

Expert quality controls are a development requirement for a trustworthy reference program. The presence of an “expert” video alone does not make it a canonical reference. Counts in illustrative manifests, such as 18 athletes and 91 recordings, are examples rather than dataset size commitments.

Proposed publication workflow

1. Ingest recordings with sport, movement, view, dominant side, and contributor metadata.
2. Check recording quality and landmark visibility, then annotate phases and execution quality.
3. Have a qualified coach review the intended biomechanical interpretation and disputed labels.
4. Build the reference indexes and run golden-set, isolation, and regression evaluations.
5. Publish a versioned manifest only after review; retain the prior release for reproducibility and rollback.

Feature design

DAT02. Reference vectors should support angles, angular velocity, body orientation, hip and shoulder rotation, torso lean, relative landmark geometry, and center displacement. Each movement defines relevant features and weights. Do not limit the architecture to the current set of eight angles.

Proposed clarification. Dataset and feature-schema compatibility should be checked before a session starts. Publication criteria, review ownership, annotation agreement, and permitted reuse of recordings remain open decisions.

10 Shared analysis and scoring

ANA01. Shared analysis services shall be reusable by movement engines while accepting movement-specific feature selections, parameters, and policies.

Service	Responsibility	Movement-specific input
ReferenceMatcher	Weighted nearest-neighbour comparison inside the allowed reference domain	Features, weights, sport, movement, phase, and optional context filters
TrajectoryScorer	Dynamic Time Warping comparison of athlete and reference trajectories; report similarity and confidence	Relevant joints or features and score contribution
KineticChainValidator	Evaluate segment activation peaks and ordering	Expected chain and applicable segment vocabulary
FluidityScorer	Offer jerk-based fluidity as one available metric	Whether fluidity applies and its contribution
ActionQualityModel	Run model inference on completed rep trajectories	Sport, movement, compatible model, and trajectory

The kinetic-chain registry should support hips, torso, shoulders, elbows, wrists, knees, and ankles. A movement may define a sequence such as hips → torso → shoulder → elbow → wrist. Available observations determine which segments can be evaluated reliably.

Score policies

SCO01. Overall scoring shall be configuration-driven by movement. Preserve overall score, component scores, and confidence even when the interface displays a single headline value. Each movement decides whether and how phase quality, trajectory, kinetic chain, extension, balance, fluidity, and AQA contribute.

No universal percentage for fluidity or common joint-weight table should be assumed. Example configurations are provided in Appendix D; those values require sport-specific validation before production use.

Post-repetition model behavior

ANA02. ST-GCN and other expensive AQA inference must run asynchronously after rep completion, outside the live frame path. ActionQualityModel provides a stable interface that may use different models for different sports.

ANA03. If the model is unavailable, production must return `aqa_status: "unavailable"` and `aqa_score: null`. Randomized or fabricated scores are prohibited.

Proposed clarification. Scoring policies should explicitly define missing-component handling and finalization. A late AQA result must attach to the correct rep and disclose whether it changes the overall score. Until this policy is agreed, an unavailable component must remain visibly unavailable.

11 Coaching and confidence

The Coaching Engine converts movement analysis, phase, detected errors, severity, and available athlete history into prioritized feedback. Raw detection and score calculation remain separate from athlete-facing language.

COA01. Coaching should return a primary correction, an optional secondary correction, a positive observation where supported, and confidence. Live feedback should contain one primary correction and no more than one secondary correction.

Correction order

1. Safety-related issue supported by the evidence.
2. Major mechanical issue.
3. Major performance issue.
4. Persistent recurring issue.
5. Minor optimization.

Rules remain specific to sport and movement. A rule identifies its condition, severity, message, and actionable correction. The sample hip-lead rule in Appendix D illustrates the format, not a validated timing threshold.

Confidence gates

Evidence limitation	Required feedback behavior
Low tracking confidence, occlusion, or incomplete body visibility	Suppress precise biomechanical correction; provide setup or tracking guidance.
Unclear movement	Ask the athlete to perform again; avoid unsupported movement-specific conclusions.
Uncertain phase	Avoid phase-specific correction until phase evidence is sufficient.
Weak reference confidence or incompatible camera context	Limit feedback to supported general observations.
Missing model or reference-dependent result	Identify the unavailable component; do not fabricate a score.

COA02. The system shall support abstention when evidence is insufficient. Poor lighting, occlusion, unsupported views, phase ambiguity, and uncertain movement can all lower confidence. Confidence thresholds and recovery rules require calibration.

COA03. Stabilize issues across multiple frames using candidate, confirmed, resolved, and persistent states. A single noisy frame must not generate flickering corrections.

Proposed clarification. Store the issue identifier, rule version, supporting phase, severity, confidence, and affected rep so feedback can be reviewed. When precision is limited, language should reflect that limitation, for example, “Your hip rotation appears to be starting late.”

12 Training flow and failure handling

The live journey begins with explicit context and validates evidence before specific feedback. Tracking guidance remains useful even when detailed scoring is unavailable.

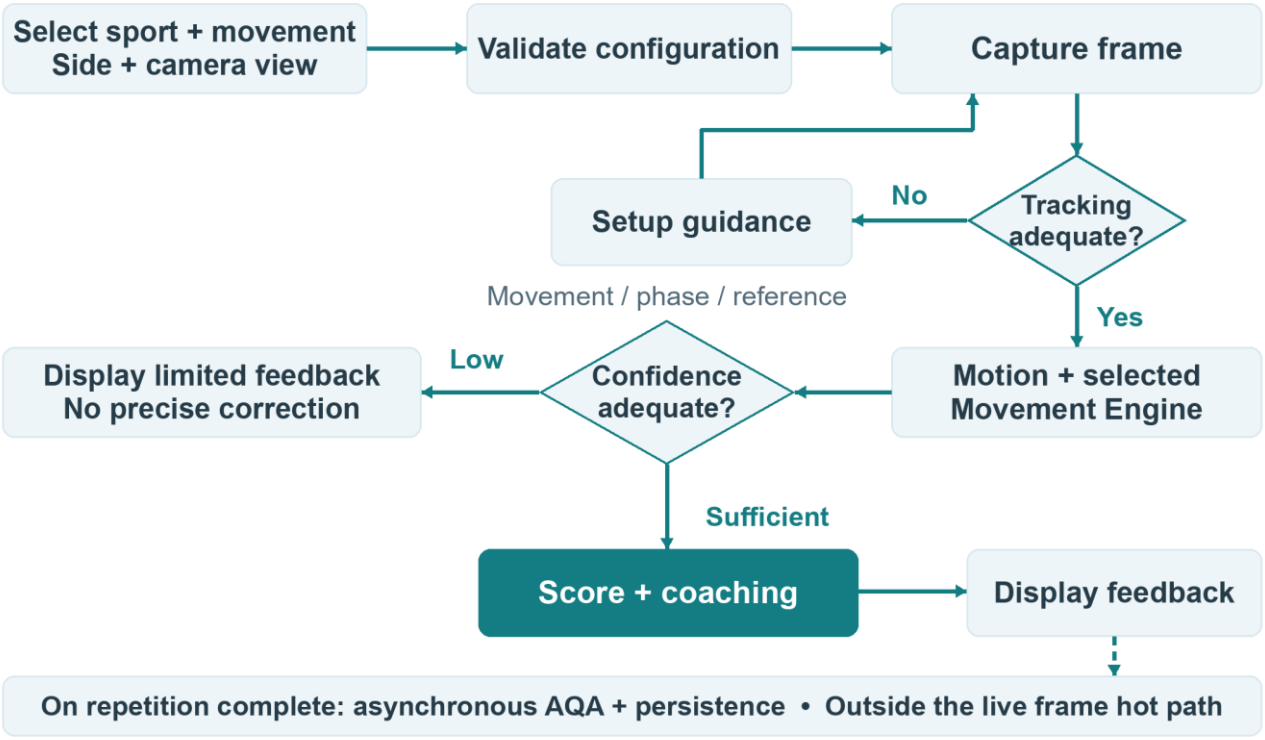


Figure 4 Live training flow with confidence gates and separate post-rep processing

Proposed failure and recovery behavior

Condition	Expected response
Unsupported or disabled sport or movement	Reject session setup with a specific reason; do not substitute another engine.
Required feature, model, or eligible reference missing	Expose the affected capability as unavailable; allow only the analysis that remains valid.
Temporary tracking loss	Suppress precise corrections; guide setup; avoid inventing a completed rep.
AI or connection interruption	Show interrupted analysis, stop presenting stale feedback as live, and apply an explicit reconnect policy.
Stop, cancellation, or duplicate completion	Finalize or cancel predictably and avoid duplicate rep records or training rewards.

These recovery details are proposed additions to the source requirements. Timeout values, reconnect behavior, invalid-rep rules, score finalization, and retry limits must be decided before implementation is accepted.

13 Real time performance

PER01. Retain the backend's single in-flight frame and latest pending frame strategy. When a new frame arrives during analysis, it replaces the older pending frame. Completion releases the newest pending frame for processing.

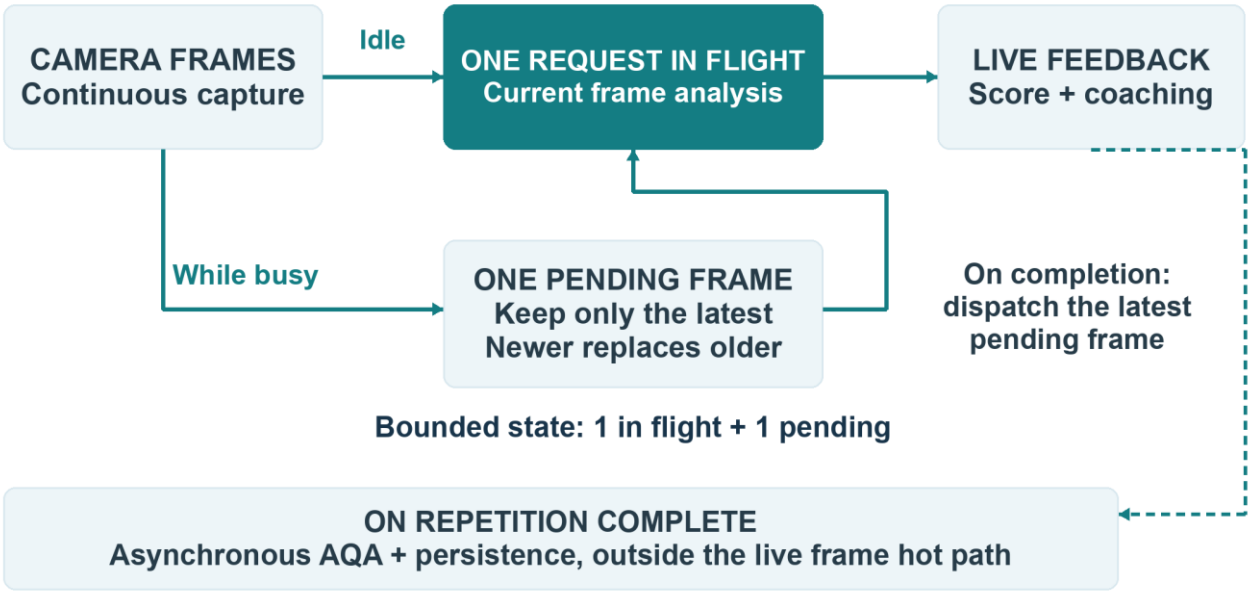


Figure 5 Bounded live frame processing and asynchronous repetition analysis

Measure	Target or constraint
AI analysis P50	Less than 60 ms
AI analysis P95	Less than 120 ms
Capture to displayed feedback	Less than 150 ms where the environment permits
Live queue	One in-flight frame plus the latest pending frame; no unlimited queue
Post-rep models	AQA and expensive inference remain outside the live frame path

PER02. Measure MediaPipe, network, backend relay, core motion processing, sport-engine processing, reference lookup, response serialization, and frontend rendering separately. Track end-to-end timing as well as the individual stages.

Proposed benchmark definition. Record hardware, browser, camera resolution and frame rate, network conditions, movement, dataset size, and concurrent sessions for each performance report. Select the end-to-end percentile and the acceptable regression tolerance before using the benchmark as a release gate. The targets above are requirements, not measured performance results.

14 APIs and integration

API01. Preserve the existing Socket.IO event names and extend their contracts compatibly. Session context must travel from the frontend through the backend to the selected analysis engine.

Event	Purpose and V2 contract detail
session:start	Initialize sessionId, sport, movement, analysisMode, dominantSide, and cameraView.
frame:submit	Submit the pose observation associated with the active session. Preserve frame timing, identity, and available depth.
feedback:result	Return phase, rep state, scores and confidence, issues, metrics, tracking status, and engine diagnostics.
session:stop	Stop analysis and finalize session processing according to the lifecycle policy.

Proposed V2 service endpoints

Method and path	Responsibility
POST /v2/analyze/frame	Analyze a live observation using the session's selected engines.
POST /v2/analyze/rep	Analyze a completed trajectory and rep-level components.
POST /v2/session/start	Validate configuration and initialize analysis state.
POST /v2/session/end	Finalize analysis and release session resources.
GET /v2/sports	Discover available sports.
GET /v2/sports/{sport}/movements	Discover movements supported by a selected sport.
GET /v2/engines/status	Expose engine availability and status.
GET /v2/health	Report service health.

API02. Keep `/analyze` and `/analyze/dynamic` operational during migration. The dynamic route can delegate through LegacyDynamicAdapter to SportEngineRegistry. Move the frontend to V2 contracts after compatibility is verified.

Contract completion

Appendices A and B retain the proposed input and result shapes. Before implementation, define shared types, naming conventions, valid ranges, nullability, schema versioning, error responses, and unsupported-capability behavior. In particular, the sample result uses numeric scores and optional diagnostics, while abstention and release diagnostics require explicit representation.

Proposed clarification. Validate that the authenticated user owns the session on every session and frame operation. Reject a frame whose context does not match its initialized session; do not silently reroute it.

15 Persistence and athlete intelligence

DB01. Preserve User, TrainingSession, PoseFrame, UserStats, EnhancedStreak, Achievement, Leaderboard, and Circle. Introduce or extend Sport, Movement, MovementVersion, ReferenceDatasetVersion, RepAnalysis, MetricResult, and DetectedIssue for structured analysis. AthleteMovementProfile and MovementProgress support the later P2 personalization scope.

Repetition record

RepAnalysis should capture one completed repetition without requiring reconstruction from raw frames.

Field group	Data to preserve
Identity and context	id, session_id, rep_number, sport, movement, movement_version, reference_dataset_version
Time	start_timestamp and end_timestamp
Scores	overall_score, score_confidence, phase_scores, component_scores, trajectory_score, kinetic_chain_score, fluidity_score, aqa_score
Feedback	Structured issues and associated metrics

Proposed relationship model. A TrainingSession contains multiple RepAnalysis records; each rep references the movement and dataset versions used. MetricResult and DetectedIssue records belong to a rep. AthleteMovementProfile and MovementProgress aggregate comparable results for one athlete and movement.

Configurable frame persistence

Mode	Storage behavior
Debug	Store every analyzed frame.
Production normal	Store sampled frames, phase-transition frames, issue frames, and rep trajectories.
Research	Store complete trajectories.

DB02. Persistence mode must be configurable by environment or session. Retention duration and access policy are separate decisions; choosing a storage mode does not imply indefinite retention.

Personalization progression

Foundation work should capture structured rep metrics. P2 Athlete Intelligence can use phase scores, kinetic-chain metrics, recurring issues, average and best scores, consistency, and trends to personalize future sessions.

For example, an illustrative set of ten smash reps may contain late hip initiation in eight, low elbow position in three, and slow recovery in five. The recurring-issue layer may prioritize hip initiation. These counts are an example of aggregation, not observed athlete results.

Proposed clarification. Trends should distinguish genuine performance changes from changes in scoring policy, model, or reference version. Cross-version comparison needs a defined compatibility or recalibration rule.

16 Session state and operations

OPS01. Analysis must depend on SessionStateStore rather than directly on Python process memory. Use InMemorySessionStateStore for development and target RedisSessionStateStore for production scaling.

Each AnalysisSession holds core state, sport session, and movement session. This includes the selected engines, phase state, signal filters, trajectory, kinetic state, rep counter, issue history, and score history.

Observability and provenance

OPS02. Log session latency, frames per second, tracking confidence, selected sport and movement engines, phase transitions, reference-search time, scoring time, errors, and dropped frames. Track P50, P95, P99, throughput, AI service failures, phase confidence, and movement confidence when classification is available.

OPS03. Core and sport-engine versions must appear in foundation diagnostics. The longer-term provenance contract should identify core, sport, movement, reference dataset, model, and scoring policy versions. Dataset version recording is already required for reproducible scoring.

Proposed operational requirements

Area	Proposed implementation requirement
Session lifecycle	Define start, active, interrupted, finalizing, stopped, and expired behavior; release filter and trajectory state on termination.
Concurrency	Give each session a clear processing owner; serialize state updates and prevent duplicated rep completion.
Reconnects	Specify whether state can resume, when a new session is needed, and how stale or out-of-order frames are rejected.
Version compatibility	Validate serialized state and engine versions before restoring a session; avoid mixing incompatible state during deployment.
Access control	Authenticate session operations and enforce session ownership; restrict research data and diagnostics to authorized roles.
Data handling	Define consent, retention, deletion, access, export, encryption, and research reuse rules; exclude sensitive media and identifiers from routine diagnostics.

These additions describe implementation decisions that need review. This PRD does not assert a compliance certification or prescribe a legal retention period.

Service degradation

Proposed behavior is to expose engine and model availability through status endpoints, distinguish unavailable analysis from a valid low score, and keep transient failures observable. A failed post-rep model must not stall live frame processing. Availability targets, capacity limits, state expiry, and operational alert thresholds remain open.

17 Migration and rollout

Taekwondo is the first Sport Engine because the existing platform includes ten dynamic movements, phase definitions, reference infrastructure, trajectory scoring, AQA, and validation work. Migrate those assets before using a second sport to test extensibility.

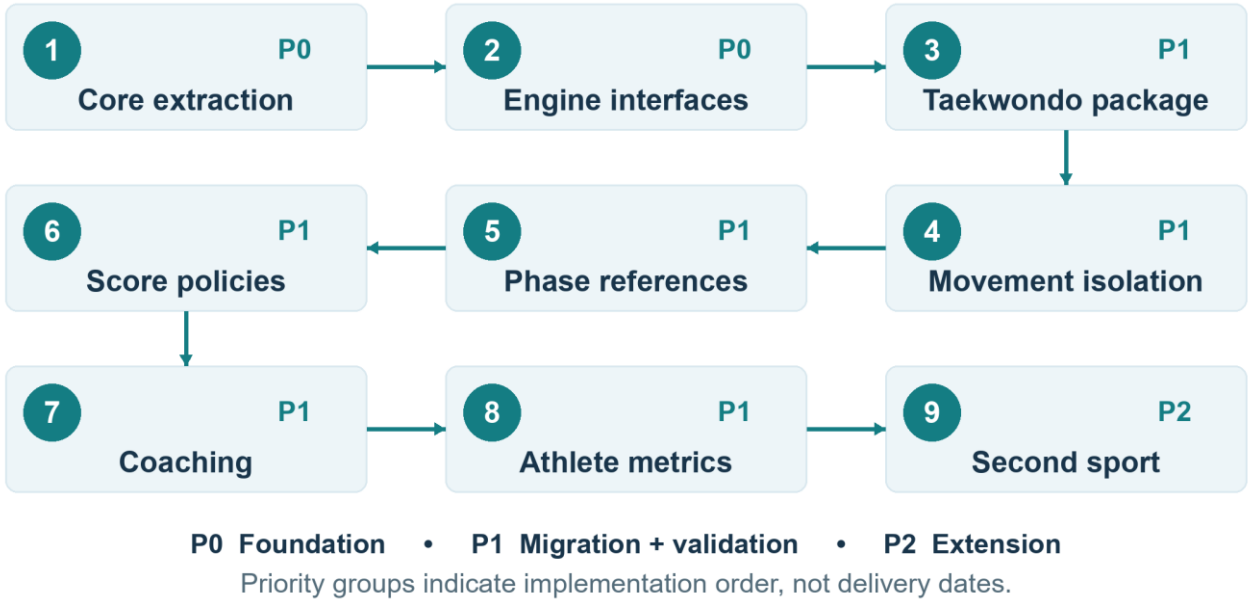


Figure 6 Migration sequence from shared core extraction to a second native sport

Migration constraints

Core extraction must preserve behavior. Introducing the engine interfaces should wrap the current pipeline without major scoring changes. Taekwondo movement definitions, weights, references, phases, and AQA then move into its package. Separate movement engines, phase-aware reference extraction, movement score policies, and coaching follow in that order.

Structured rep metrics prepare Athlete Intelligence. The second sport must use the plugin interface without modifying the core except for genuinely universal features. Its selection and delivery timing remain open.

Proposed rollout and rollback

1. Establish reproducible baseline results and versioned golden trajectories before changing routing.
2. Compare the adapter and modular implementations using the same recorded inputs and documented tolerances.
3. Enable the migrated sport for controlled validation after isolation, correctness, latency, and compatibility checks pass.
4. Expand availability after coach review and operational evidence meet the agreed gates.
5. Revert routing to the last validated engine and compatible dataset if isolation, scoring integrity, or performance fails. Preserve affected result versions for investigation.

No calendar dates or assigned delivery owners are established in this PRD. Rollout cohorts, rollback triggers, and operational ownership are proposed decisions for release planning.

18 Validation and success metrics

VAL01. Each movement engine requires unit tests, recorded trajectory tests, regression tests, and coach-reviewed athlete validation. A visually plausible output alone is insufficient evidence.

Golden movement dataset

Create a permanent, human-annotated validation set for each movement with 20 correct, 20 moderate, and 20 intentionally incorrect examples. Expand the set over time. Evaluation should measure phase accuracy, rep detection, issue detection, scoring agreement, false positives, and false negatives.

Proposed evaluation control. Keep validation examples separate from reference construction and model training where possible, and document athlete, view, dominant-side, and execution-quality coverage. This reduces the risk of measuring memorization rather than useful movement analysis.

Area	Metrics and evidence
Recognition when enabled	Precision, recall, and F1 for sport or movement classification
Phase detection	Phase accuracy and transition timing error
Issue detection	Precision, recall, false-positive rate, and false-negative rate
Rep detection	Rep-count accuracy on known trajectories
Scoring	Pearson and Spearman correlation with coach scores; mean absolute error
Reliability	Consistency across repeated similar executions and compatible capture conditions
Performance	P50/P95/P99 latency, throughput, dropped frames, and per-stage timing

Test coverage

- Unit tests cover angle calculations, trigger logic, score calculation, and reference filtering.
- Recorded trajectories assert expected phases, rep counts, and major issues.
- Regression tests identify unexpected score changes after an engine, model, policy, or dataset update.
- Athlete tests include correct execution and known incorrect executions reviewed by a qualified coach.
- Isolation tests instrument repository access and verify that unselected sports receive zero accesses.

VAL02. Define pass thresholds for accuracy, score agreement, reliability, and regression tolerance before release approval. The source specifies the measures and latency targets but does not establish numerical quality thresholds. Record the dataset versions, environment, engine versions, and review outcome with each evaluation.

19 Release acceptance

Foundation acceptance requires the following checks to pass with recorded evidence. Product, engineering, and coaching approvers and numerical quality thresholds remain to be assigned.

ID	Acceptance criterion	Evidence
AC01	Existing Taekwondo analysis runs through TaekwondoEngine.	End-to-end session and regression results
AC02	A common motion pipeline standardizes 2D/3D data and preserves available z.	Contract and normalization tests
AC03	Core angles, velocities, smoothing, and confidence work independently of sport; no Taekwondo-specific logic remains in core.	Package review and unit tests
AC04	Sport and movement references are isolated, and phase-specific matching is supported.	Retrieval tests and repository-access instrumentation
AC05	Each Sport Engine declares movements, owns its biomechanical rules and references, supports independent versioning, and can be disabled independently.	Registry, configuration, and disablement tests
AC06	Movement engines implement phases, rep detection, references, score policy, and coaching rules.	Movement unit tests and golden-set validation
AC07	Scoring is configurable by movement; coaching rules are separate from raw scoring.	Policy and coaching tests
AC08	Core and sport versions appear in diagnostics; sessions retain producing dataset versions.	Result and persistence inspection
AC09	The existing frontend receives real-time feedback; authentication, gamification, and existing routes continue working.	Integration and compatibility checks
AC10	Live analysis stays within agreed latency targets without material regression; AQA stays outside the live path.	Benchmarks and queue instrumentation
AC11	Production never returns randomized AQA scores. Missing models return unavailable and null.	Missing-model test
AC12	Low-confidence behavior abstains or limits correction, and feedback remains stable.	Confidence and noisy-input tests

Mandatory isolation scenario

Start a session with sport set to badminton in the isolation test fixture. Assert that tennis, Taekwondo, and cricket reference-repository access counts are each zero. Apply the equivalent test to every registered sport. This test validates architecture; it does not imply that badminton is already a released sport.

Release evidence package. Retain automated test results, golden-set and coach review outcomes, performance benchmarks, version manifests, unresolved exceptions, and the release decision. A failed mandatory isolation or fabricated-score check prevents acceptance.

20 Risks and open decisions

Risk	Impact	Mitigation
False precision	Precise-looking measurements may exceed reliable visual evidence.	Confidence, visibility and view gates, and uncertainty-aware feedback.
Weak reference data	Bad labels or recordings reduce scoring quality.	Coach validation, controlled capture, multiple athletes and views, and immutable versions.
Overgeneralization	One algorithm or policy misinterprets sport-specific mechanics.	Isolated Sport and Movement Engines with validated policies.
Excessive specialization	Repeated generic logic increases maintenance and divergence.	Shared motion and analysis services with explicit contracts.
Reference growth	Brute-force matching becomes too slow.	Hierarchical and phase filtering, measurements, then replaceable indexes.

Decisions required before implementation acceptance

ID	Decision	Suggested reviewing function
D01	Product owner, technical owner, coaching reviewer, launch scope, milestones, and second sport	Product and engineering
D02	Coordinate units, normalization, derivative timing, feature IDs, per-feature confidence, and missing-value conventions	AI and platform engineering
D03	Confidence thresholds, score calibration, no-reference behavior, missing-component policy, and AQA finalization	AI and coaching
D04	Required provenance fields, schema versions, error responses, and camera-view vocabulary	Platform engineering
D05	Benchmark environment, workload, end-to-end percentile, and material-regression tolerance	Engineering and quality
D06	Accuracy, reliability, coach-agreement, and golden-set pass thresholds	AI, quality, and coaching
D07	Session timeouts, reconnects, serialization, concurrency ownership, and state cleanup	Backend and AI engineering
D08	Upload formats and limits, processing experience, retention, consent, deletion, and access controls	Product and platform

Suggested reviewing functions are proposed responsibilities, not assigned people or confirmed approvals. Resolve foundation decisions before claiming release readiness; later-phase decisions can remain attached to their planned capability.

Appendix A Motion contracts

These representative TypeScript contracts preserve the proposed data shape. Shared types such as NormalizedLandmarks and Vector3 require final definitions, units, and coordinate conventions. Per-feature confidence and unavailable measurements need an explicit schema decision before implementation.

```
interface Landmark {
  name: string;
  x: number;
  y: number;
  z?: number;
  visibility: number;
}

interface RawPoseObservation {
  timestampMs: number;
  frameId: string;
  landmarks: Landmark[];
  detectionConfidence: number;
  source: { width: number; height: number; fpsEstimate: number };
}

interface MotionFrame {
  timestampMs: number;
  landmarks: NormalizedLandmarks;
  jointAngles: Record<string, number>;
  jointAngularVelocity: Record<string, number>;
  jointAngularAcceleration: Record<string, number>;
  positionVelocity: Record<string, Vector3>;
  body: {
    torsoLean: number;
    shoulderRotation: number;
    hipRotation: number;
    shoulderHipSeparation: number;
    centerOfBody: Vector3;
  };
  orientation: {
    vertical: boolean;
    facingDirection?: string;
    dominantSideEstimate?: "left" | "right" | "unknown";
  };
  confidence: { tracking: number; visibility: number };
  motionState: { active: boolean; onsetProbability?: number };
}
```

Feature Registry examples

Illustrative identifiers include `joint_angle.right_knee`, `joint_velocity.right_knee`, `torso.rotation`, `hip.rotation`, `shoulder.rotation`, `center.vertical_velocity`, and `body.orientation`. An engine declares its required subset. Standardize naming rather than maintaining parallel dotted and underscored forms.

Appendix B Session and result contracts

The example below illustrates a badminton session. It does not commit badminton to the foundation release. The analysis result is a representative contract and needs the missing-value and diagnostics decisions described in D03 and D04.

```
{
  "sessionId": "example-session",
  "sport": "badminton",
  "movement": "smash",
  "analysisMode": "dynamic",
  "dominantSide": "right",
  "cameraView": "front"
}
```

```
interface ApexAnalysisResult {
  sessionId: string;
  sport: string;
  movement: string;
  phase: { name: string; index: number; confidence: number };
  rep: { count: number; completed: boolean };
  scoring: {
    overall: number;
    confidence: number;
    components: Record<string, number>;
  };
  issues: CoachingIssue[];
  metrics: Record<string, number>;
  tracking: { confidence: number; bodyVisible: boolean };
  diagnostics?: {
    engineVersion: string;
    sportEngineVersion: string;
    movementEngineVersion: string;
    referenceDatasetVersion: string;
  };
}
```

Proposed contract completion. Define CoachingIssue and unavailable-score states; make foundation-required diagnostics mandatory in the production contract. Add model and scoring-policy version fields when those components participate in analysis. The optional sample diagnostics block must not override release acceptance requirements.

Future equipment observation

```
interface TrackedSportObject {
  type: string;
  position: Vector3;
  velocity?: Vector3;
  confidence: number;
}
```

Appendix C Package and code organization

This recommended layout expresses ownership boundaries. It is a target organization, not a claim about the current repository. Package names and reference storage locations should be standardized during implementation.

```
apps/ai-service/app/
  core/
    perception/
    motion/
      normalization.py   angles.py       velocity.py
      acceleration.py    orientation.py  motion_frame.py
    signal/
      one_euro.py  butterworth.py  derivative.py  interpolation.py
    features/
      registry.py  extractor.py
    confidence/
      tracking_quality.py
  sports/
    base/
      sport_engine.py  movement_engine.py  contracts.py
    taekwondo/
      engine.py  movements/  data/  models/  rules/
    badminton/
      engine.py  movements/  data/  models/  rules/
    tennis/
    cricket/
  analysis/
    phase/  matching/  trajectory/  kinetic_chain/
    scoring/  coaching/
  athlete/
    profiling/  trends/  weakness_detection/  personalization/
```

Sport and movement packages

A sport package should contain `sport.yaml`, `movements/`, `references/`, `rules/`, `models/`, and `tests/`. The manifest declares the sport, package version, supported movements, and required core features.

```
sports/badminton/movements/smash/
  definition.yaml
  phases.yaml
  scoring.yaml
  kinetic_chain.yaml
  coaching_rules.yaml
  references/
  tests/
```

Taekwondo engines include ApChagiEngine, DollyoChagiEngine, YeopChagiEngine, DwiChagiEngine, and the remaining existing movements. Each movement receives phase definitions, references, feature weights, kinetic chain, trajectory configuration, score policy, and coaching rules.

Appendix D Illustrative policies and glossary

All values on this page are examples from the proposed design. They are not validated production weights, measured athlete performance, or acceptance thresholds.

Example movement policies

Example	Components and illustrative weights
Ap chagi score	Phase quality 0.30; trajectory 0.25; kinetic chain 0.20; extension 0.15; balance 0.10
Taekwondo feature emphasis	Hip rotation 1.8; knee extension 2.0; support knee 1.2; shoulder position 0.5
Badminton smash feature emphasis	Shoulder rotation 1.7; elbow extension 1.5; hip rotation 1.4; torso rotation 1.4; wrist or forearm signal is future work

An illustrative score can preserve overall 82, phase accuracy 87, trajectory 79, kinetic chain 66, fluidity 91, balance 84, and confidence 0.92. This is a payload example; it does not assert that these components produce 82 under the Ap chagi formula above.

```
{
  "rule_id": "badminton.smash.hip_lead",
  "condition": { "shoulder_peak_before_hip_peak_ms": "> 80" },
  "severity": "medium",
  "message": "Your shoulder is initiating before your hips.",
  "correction": "Start hip rotation, then accelerate the shoulder."
}
```

Glossary

Term	Meaning in this PRD
AQA	Action Quality Assessment; a model-based evaluation of completed movement.
DTW	Dynamic Time Warping; sequence alignment used for trajectory comparison.
FSM	Finite-state machine; the configurable phase-transition framework.
ST-GCN	Spatial Temporal Graph Convolutional Network used by the current AQA capability.
MotionFrame	Sport-neutral motion features and confidence for one observed time point.
Golden dataset	Permanent annotated examples used to evaluate movement analysis.
Semantic contamination	Using another sport’s or movement’s references or rules to interpret an execution.
P50 and P95	Latency values at the 50th and 95th percentiles of measured requests.